

PINTOS

Alarm Clock

Implementação4|hmrn|20desetembro de 2026

Entrega: hmrn.pdf e hmrn.tar. O .tar contém somente os arquivos de fonte modificados em relação ao Pintos original. hmrn_espelho_tar reproduz o .tar e hmrn_espelho2_github contém a árvore completa para o GitHub. As seções 1 a 7 estão presentes; a seção 7 fica reservada ao usuário.

1. Objetivo

Substituir a espera ocupada de timer_sleep por suspensão da thread.
Garantir espera de pelo menos o número solicitado de ticks.
Entregar o kernel, os testes, os registros de execução e este relatório.
Disponibilizar sete etapas cumulativas e a versão final para comparação.

2. Checklist de requisitos

Requisito	Status	Como testar / evidência
Alarm Clock sem busy wait	OK	alarm-blocking: thread BLOCKED; timer.c sem laço de yield em timer_sleep.
Não retornar antes do prazo	OK	alarm-single, multiple, simultaneous e alarm-deadline.
Zero e valores negativos	OK	alarm-zero, alarm-negative e INT64_MIN no alarm-deadline.
Sincronização com o timer	OK	Semáforo por espera; seção crítica contra ISR; stress com 480 suspensões.
TIMER_FREQ e wrappers	OK	Frequência 100; msleep/usleep/nsleep preservados em relação à base.
Compilação em threads e limpeza	OK	make clean; make -j4; etapa7.log.
Arquivos modificados no .tar	OK	10 arquivos fonte alterados em relação ao Pintos original.
Árvore completa para GitHub	OK	hmrn_espelho2_github contém 636 arquivos de fonte.
Progressão em sete pastas	OK	pasta1 a pasta7, cada uma com fonte e evidências do marco.
GitHub e seção 7	Usuário	GitHub será feito depois, conforme orientação na conversa.

Escopo funcional: Alarm Clock. Prioridades, doação e MLFQS não foram pedidos no enunciado fornecido. O diagnóstico alarm-priority falhou e está documentado na seção 6.

3. Como reproduzir

Ambiente observado: Ubuntu WSL2, GCC 15.2.0, GNU Make e QEMU 10.2.1 para i386. A última execução registrou QEMU com pacote 1:10.2.1+ds-1ubuntu3.2. Os verificadores do Pintos utilizam Perl.

Dependências e execução

```
sudo apt-get update
sudo apt-get install build-essential perl qemu-system-x86
git clone https://github.com/jhu-cs318/pintos.git pintos
tar -xf hmrn.tar
cp -a hmrn/src/. pintos/src/
cd pintos/src/threads
make clean
make -j4
```

O .tar é um conjunto de alterações, por isso os arquivos devem ser aplicados sobre a base do Pintos. hmrn_espelho2_github já oferece a árvore completa equivalente para publicar no GitHub. No Windows, execute os comandos pelo Ubuntu/WSL. Não há executável .exe.

Alvo	O que executa
make -C src/threads	Compila o Pintos Threads, incluindo o helper de execução.
make -C src/threads clean	Remove os arquivos compilados; preserva os fontes.

Teste isolado e novo registro

```
export PATH="$PWD/src/utils:$PATH"
cd src/threads/build
pintos -v -k -T 20 --qemu -- -q run alarm-stress
```

A suíte padrão make check do Pintos inclui exercícios além desta entrega. Para conferir somente um caso, use o comando isolado acima. O runner pode encerrar com código zero após PANIC; a evidência deve considerar a mensagem produzida.

4. Arquitetura resumida

Arquivo	Responsabilidade específica
src/devices/timer.c	Fila ordenada, semáforo por espera, bloqueio e despertar por interrupção.
src/lib/arithmetic.c	Helper __udivmoddi4 requerido pelo GCC moderno com otimização.
src/Make.config	Compilação 32-bit, freestanding, GNU99, O2, sem PIE; frame pointers.
src/threads/Make.vars	Seleciona QEMU como emulador.
src/tests/threads/alarm-blocking.c	Inspeciona BLOCKED e espera a thread concluir.
src/tests/threads/alarm-extra.c	Prazos, carga, extremos, contrato inválido e helper aritmético.
tests.c / tests.h / Make.tests	Registram e compilam os testes adicionais; verificadores oficiais preservados.
src/utils/setitimer-helper	Helper de execução adaptado para o ambiente de testes.

Decisões técnicas e consequências

Semáforo local: cada espera tem um registro na pilha da thread. sema_down bloqueia; sema_up no timer a devolve à fila ready. O registro só sai da pilha após o retorno, e o handler o remove da lista antes de sinalizar. Dispensa alocação dinâmica e mudanças em struct thread.

Fila ordenada: a inserção custa $O(n)$. Um tick sem vencimentos consulta só o primeiro prazo, $O(1)$; k despertares custam $O(k)$. Todos os prazos vencidos são removidos na mesma interrupção. A etapa 3 percorria a fila inteira por tick, $O(n)$.

Atomicidade: leitura do relógio, inserção e sema_down ocorrem com interrupções desativadas. A troca de thread permite ao próximo contexto habilitá-las. O estado original é restaurado no retorno. Isso protege dados compartilhados com a ISR sem adquirir um lock dentro dela.

Prazo sem sinal: wake_tick usa uint64_t para que ticks + INT64_MAX não cause overflow assinado imediato. O contador global int64_t original mantém seu limite de longo prazo; não foi implementado suporte ao wrap desse contador.

Integração: threads/synch.c e thread.c são usados sem alterações. A função timer_sleep não contém espera ocupada. As esperas curtas de calibração e das funções de delay da base continuam preservadas, conforme a orientação de não modificar os wrappers.

5. Estratégias e diário de desenvolvimento

5.1 Estratégias da sessão

ID	Abordagem
S1	Estabelecer a base e um teste que revele busy wait.
S2	Bloquear por semáforo usando fila não ordenada.
S3	Ordenar a fila, validar limites e submeter testes a defeitos deliberados.

5.2 Diário de tentativas reais

Quando ¹	Estr.	Tentativa	Resultado	Hipótese/causa ou evidência
15:12:04	S1	Base original compila, mas reinicia durante o boot.	Falha	Hipótese: incompatibilidade da configuração antiga com GCC atual; causa isolada não demonstrada.
15:12:52	S1	Ajustar flags e reconstruir.	Falha	GCC gerou chamada ausente a __udivmoddi4.
15:13:23	S1	Modo freestanding e helper aritmético.	OK	Cinco testes passam; busy wait ainda existe.
15:14:52	S1	Adicionar casos limite e detector de bloqueio.	Esperada	Teste acusa READY: timer_sleep ainda usa thread_yield.
15:15:55	S2	Fila não ordenada e semáforo por espera.	OK	Cinco oficiais + alarm-blocking passam.
15:17:40	S3	Ordenar prazos e despertar todos os vencidos.	OK	Mesma suíte passa.
15:18:51	S3	Testar concorrência, extremos e usos inválidos.	OK	10 positivos e 2 negativos verificados.
15:19:33	S3	Compilar três defeitos isoladamente.	Esperada	busy_wait, early_wakeup e lost_wakeup detectados.
15:19:56	S3	Executar alarm-priority como diagnóstico.	Falha	Escalonador base round-robin; prioridade fora do escopo.
15:21:03	S3	Recompilar do zero e repetir a suíte final.	OK	10 positivos e 2 negativos verificados.

6. Erros, testes e evidências

6.1 Matriz de validação

Teste / situação	Resultado	O que foi observado
alarm-single	PASS	5 threads, uma espera por thread.
alarm-multiple	PASS	Esperas repetidas e ordem por prazo.
alarm-simultaneous	PASS	3 threads, 5 rodadas com prazo comum.
alarm-zero / alarm-negative	2 PASS	Durações não positivas retornam.
alarm-blocking	PASS	Estado BLOCKED durante a espera; conclusão depois.
alarm-deadline	PASS	6 prazos; zero, -1 e INT64_MIN; interrupções restauradas.
alarm-stress	PASS	24 threads x 20 rodadas = 480 esperas, sem retorno antecipado.
alarm-huge	PASS limitado	INT64_MAX permanece bloqueado durante 3 ticks observados.
alarm-arithmetic	PASS	4 casos de divisão/resto, incluindo números de 64 bits.
Interrupções desligadas	Erro esperado	ASSERT em timer_sleep; mensagem conferida pelo script.
Nome de teste inexistente	Erro esperado	PANIC no despachante: no test named.
Mutação busy_wait	Detectada	alarm-blocking acusa estado READY.
Mutação early_wakeup	Detectada	alarm-deadline acusa retorno antes de 2 ticks.
Mutação lost_wakeup	Detectada	Não conclui; timeout registrado após 6 s no runner -T 5.
alarm-priority (diagnóstico)	FAIL	Ordem por prioridade não implementada no escalonador base.

Total da aceitação: **10 testes positivos passaram e 2 usos inválidos foram rejeitados**. Os 3 defeitos deliberados foram detectados em execuções separadas. O diagnóstico de prioridades não integra esse total e sua falha não foi ocultada.

Fontes consultadas: evidencias.log preservam os comandos, horários e saídas usados nesta matriz. O .tar final contém somente os arquivos fonte alterados.

6.2 Evidências visuais dos registros

Os quadros abaixo são transcrições das saídas reais preservadas em evidencias.log. Não são saídas simuladas. Os números de linha permitem conferir a origem.

Erro que demonstra o problema original

```
(alarm-blocking) sleeping thread status=1; expected BLOCKED=2
(alarm-blocking) FAIL: timer_sleep keeps the sleeper READY
```

```
(busy waiting)
```

evidencias.log, linha 589

A mesma verificação depois da correção

```
(alarm-blocking) sleeping thread status=2; expected BLOCKED=2
(alarm-blocking) PASS
Thread: 100 idle ticks, 24 kernel ticks, 0 user ticks
```

evidencias.log, linha 685

Validação com concorrência

```
(alarm-stress) 24 threads completed 480 sleeps
                without early returns
(alarm-stress) PASS
```

evidencias.log, linha 949

Falhas de ambiente e limitações observadas

O primeiro boot reiniciava antes da inicialização completa. As mudanças de flags e do helper de runtime resolveram a execução, mas não foi feito um experimento que isolasse qual flag causava o reboot. A falta de `__udivmoddi4` foi comprovada pela mensagem de link e corrigida reutilizando `udiv64` da base.

A compilação final ainda apresenta avisos herdados sobre `__builtin_frame_address` e permissões RWX do loader. Não houve promessa de compilação sem warnings. Os trechos novos de `timer.c` e dos testes não produziram warnings nas execuções registradas.

O teste `alarm-priority` falhou por ordenar threads segundo prioridades, algo que o round-robin original não garante. O Alarm Clock garante elegibilidade após o prazo; não adiciona prioridades. O teste extremo não valida a passagem integral de `INT64_MAX` ticks.

7. Uso de IA

Utilizei a ia primeiramente para me ajudar com o estudo e entendimento geral do pint-os tirando dúvidas gerais, tive bastante problemas para verificar se eu estava fazendo certo e aplicando de forma correta o guia, assim, pedi para a IA me explicar melhor como as alterações funcionavam e o que era essencial mudar.

8. Reflexão final

Os testes de tempo da base passavam mesmo com espera ocupada.
Inspecionar o estado BLOCKED tornou esse requisito verificável.
O semáforo por espera permitiu reutilizar a sincronização existente.
Ordenar a fila reduziu o trabalho da interrupção sem vencimentos.
As mutações mostraram que os testes detectam três classes de defeito.
Uma próxima evolução deve ampliar portabilidade e limites temporais.

9. Checklist final

Conferência	Situação
Código fonte alterado	10 arquivos no hmrn.tar; aplicação sobre a base documentada na seção 3.
Árvore completa	hmrn_espelho2_github contém fontes modificadas e não modificadas.
Relatório	hmrn.pdf acompanha hmrn.tar como arquivo separado.
Sete etapas e comparação	A progressão permanece separada para comparação.
Nomes e organização	hmrn.tar, hmrn.pdf, hmrn_espelho_tar e hmrn_espelho2_github.
GitHub / seção 7	Pendências assumidas pelo usuário.

10. Se eu tivesse mais 2 horas

Executaria a mesma suíte com outra versão de GCC e QEMU.
Mediria a duração da seção crítica com centenas de sleepers.
Adicionaria um relógio controlável para testar wrap do contador.
Variaria a ordem de criação e os prazos no teste de concorrência.
Investigaria isoladamente o reboot inicial para identificar sua causa.
Revisaria os warnings herdados sem enfraquecer as verificações.

Referências e mapa da progressão

Materiais fornecidos: Implementação4 - Pint-OS(quatroimagens no ZIP); Aula 08.docx.pdf; Install PINT-OS on Linux by me.pdf; Guia_de_Relatorios_simulado.docx.pdf. O nome hmrn e a reserva da seção 7 seguem o guia fornecido.

Código base: github.com/jhu-cs318/pintos, commit `ab699959f0d3eae8e59d8507ae2474a638c3d1d3`. É a distribuição indicada pelo guia de instalação. A licença original está em `src/LICENSE`.

Referência de interface e escopo: Pintos Projects: Project 1 - Threads, Stanford. As primitivas de sincronização foram conferidas diretamente no código base. A implementação de Alarm Clock desta entrega foi escrita nesta sessão; não foi importada uma solução de aluno.

Marco	Evolução	Situação
1/7	Base compilada e testes de referencia	Registrada
2/7	Casos limite e deteccao de busy wait	Registrada
3/7	Suspensao real com semaforos	Registrada
4/7	Fila ordenada por prazo	Registrada
5/7	Testes de limites concorrencia e entradas invalidas	Registrada
6/7	Falhas deliberadas e verificacao dos detectores	Registrada
7/7	Versao completa validada e documentada	Concluída

Os marcos são uma progressão didática cumulativa, não sete frações iguais de tempo ou linhas de código. As partes 1 a 7 formam o núcleo da entrega; a parte 7 permanece reservada para preenchimento manual.

Arquivos de evidência

`evidencias.log` registra a cronologia completa das sessões do kernel. `erros.log` é a captura automática do terminal. Eles foram usados na revisão e nas transcrições da seção 6; o `hmrn.tar` final contém somente os arquivos de fonte alterados.

O trabalho foi produzido no dia indicado e não representa uma semana pessoal do usuário. O usuário solicitou a progressão, os casos errados e a entrega completa; depois informou que faria o GitHub por conta própria. Não há link remoto inventado.